

Data Preparation Guide for Pulse System

Introduction

This guide explains how to prepare and provide data to the Pulse system so that it can be ingested, analyzed, and turned into meaningful insights. Pulse accepts data in three different ways:

1. **Batch Upload** — Upload files (like spreadsheets or CSV files) directly into the system.
2. **Real-Time from a Database** — Connect Pulse directly to your database so it automatically streams new and changed data.
3. **Real-Time from an API** — Have Pulse call your API endpoint periodically to fetch new data.

The system will try to automatically map your data onto a standard structure (called the **canonical schema**). If your data does not match this structure, or if important columns are missing, the relevant analytics and insights will be skipped for those tables.

Table of Contents

1. [Canonical Schema Overview](#)
2. [Batch File Ingestion](#)
3. [Real-Time Streaming via Database URI \(CDC\)](#)
4. [Real-Time Streaming via API](#)
5. [How the Ingestion System Works](#)

1. Canonical Schema Overview

The Pulse system uses a **canonical schema** — a predefined set of tables and columns that it expects data to map onto. When you provide data, the system tries to automatically match your table and column names to this canonical structure. If a table or column does not match anything in the schema, the option of manually mapping the data is given at the end. If the manual mapping is skipped, the respective and relevant analytics will not be displayed.

The column names can be flexible but should be somewhat similar to the column names provided in the schema below so that our Natural Language Processing system is able to identify the column names.

Below is the full list of tables and their required columns:

addresses

Column	Type	Description
address_id	VARCHAR(50)	Unique ID for each address
city	VARCHAR(100)	City name (e.g. New York)
state_province	VARCHAR(100)	State or province name
postal_code	VARCHAR(20)	Zip or postal code
country	VARCHAR(100)	Full country name

customers

Column	Type	Description
customer_id	VARCHAR(50)	Unique customer ID
gender	VARCHAR(10)	Male, Female, Other, Prefer not to say
date_of_birth	DATE	Date of birth (YYYY-MM-DD)
account_status	VARCHAR(20)	Active, Inactive, Blocked
address_id	VARCHAR(50)	Foreign key to addresses
city	VARCHAR(100)	City name
state_province	VARCHAR(100)	State or province
postal_code	VARCHAR(20)	Zip or postal code
country	VARCHAR(100)	Country name
account_created_at	TIMESTAMP	When account was created
last_login_date	TIMESTAMP	Last login date
is_active	BOOLEAN	Whether account is active

suppliers

Column	Type	Description
supplier_id	VARCHAR(50)	Unique supplier ID
supplier_rating	DECIMAL(3,2)	Supplier rating score
supplier_status	VARCHAR(50)	Status (e.g. active)
is_preferred	BOOLEAN	Preferred supplier flag
is_verified	BOOLEAN	Verified supplier flag
contract_start_date	DATE	Contract start date
contract_end_date	DATE	Contract end date
city	VARCHAR(100)	City
state	VARCHAR(100)	State
zip_code	VARCHAR(20)	Zip code
country	VARCHAR(100)	Country

categories

Column	Type	Description
category_id	VARCHAR(50)	Unique category ID
category_name	VARCHAR(100)	Category name
sub_category	VARCHAR(50)	Sub-category

products

Column	Type	Description
product_id	VARCHAR(50)	Unique product ID
product_name	VARCHAR(255)	Full product name
sku	VARCHAR(100)	Stock Keeping Unit
category_id	VARCHAR(50)	Foreign key to categories
category	VARCHAR(50)	Category name
sub_category	VARCHAR(50)	Sub-category
brand	VARCHAR(100)	Brand name
supplier_id	VARCHAR(50)	Foreign key to suppliers
cost_price	DECIMAL(10,2)	Cost price
sell_price	DECIMAL(10,2)	Selling price
launch_date	DATE	Product launch date
weight	DECIMAL(8,3)	Weight
dimensions	VARCHAR(50)	Dimensions
color	VARCHAR(50)	Color
size	VARCHAR(50)	Size
material	VARCHAR(100)	Material

inventory

Column	Type	Description
inventory_id	VARCHAR(50)	Unique inventory ID
product_id	VARCHAR(50)	Foreign key to products
supplier_id	VARCHAR(50)	Foreign key to suppliers
stock_quantity	INT	Current stock quantity
reserved_quantity	INT	Reserved quantity
minimum_stock_level	INTEGER	Minimum stock level
last_restocked_date	TIMESTAMP	Last restocked date
storage_cost	DECIMAL(10,2)	Storage cost
stock_status	VARCHAR(50)	Stock status (e.g. In Stock)

wishlist

Column	Type	Description
wishlist_id	VARCHAR(50)	Unique wishlist ID
customer_id	VARCHAR(50)	Foreign key to customers
product_id	VARCHAR(50)	Foreign key to products
added_date	TIMESTAMP	Date added to wishlist
purchased_date	TIMESTAMP	Date purchased from wishlist
removed_date	TIMESTAMP	Date removed from wishlist

shopping_cart

Column	Type	Description
cart_id	VARCHAR(50)	Unique cart ID
customer_id	VARCHAR(50)	Foreign key to customers
session_id	VARCHAR(50)	Foreign key to sessions
cart_status	VARCHAR(20)	Cart status (e.g. ACTIVE)
created_at	TIMESTAMP	Cart creation time
updated_at	TIMESTAMP	Cart last updated time

cart_items

Column	Type	Description
cart_item_id	BIGINT	Unique cart item ID
cart_id	VARCHAR(50)	Foreign key to shopping_cart
product_id	VARCHAR(50)	Foreign key to products
quantity	INT	Quantity (must be > 0)
unit_price	DECIMAL(10,2)	Price per unit
total_price	DECIMAL(12,2)	Total price (auto-calculated)
added_at	TIMESTAMP	When item was added
updated_at	TIMESTAMP	When item was last updated
item_status	VARCHAR(20)	Item status (e.g. ACTIVE)

orders

Column	Type	Description
order_id	VARCHAR(50)	Unique order ID
customer_id	VARCHAR(50)	Foreign key to customers
order_status	VARCHAR(30)	Order status
subtotal	DECIMAL(10,2)	Subtotal amount
tax_amount	DECIMAL(10,2)	Tax amount
shipping_cost	DECIMAL(10,2)	Shipping cost
total_discount	DECIMAL(10,2)	Total discount
total_amount	DECIMAL(10,2)	Total amount
currency	VARCHAR(10)	Currency (e.g. USD)
order_placed_at	TIMESTAMP	When order was placed
order_shipped_at	TIMESTAMP	When order was shipped
order_delivered_at	TIMESTAMP	When order was delivered

order_items

Column	Type	Description
order_item_id	VARCHAR(50)	Unique order item ID
order_id	VARCHAR(50)	Foreign key to orders
product_id	VARCHAR(50)	Foreign key to products
quantity	INT	Quantity ordered
discount_amount	DECIMAL(10,2)	Discount applied
product_price	DECIMAL(10,2)	Product price at time of order

payments

Column	Type	Description
payment_id	VARCHAR(50)	Unique payment ID
order_id	VARCHAR(50)	Foreign key to orders
payment_method	VARCHAR(50)	Payment method
payment_provider	VARCHAR(50)	Payment provider
payment_status	VARCHAR(30)	Payment status
transaction_id	VARCHAR(255)	Transaction reference
processing_fee	DECIMAL(10,2)	Processing fee
refund_amount	DECIMAL(10,2)	Refund amount
refund_date	TIMESTAMP	Refund date
payment_date	TIMESTAMP	Payment date

reviews

Column	Type	Description
review_id	VARCHAR(50)	Unique review ID
product_id	VARCHAR(50)	Foreign key to products
customer_id	VARCHAR(50)	Foreign key to customers
rating	INT	Rating (1 to 5)
review_title	VARCHAR(255)	Review title
review_desc	TEXT	Review description
review_date	TIMESTAMP	Review date

marketing_campaigns

Column	Type	Description
campaign_id	VARCHAR(50)	Unique campaign ID
campaign_name	VARCHAR(255)	Campaign name
campaign_type	VARCHAR(50)	Campaign type
start_date	DATE	Campaign start date
end_date	DATE	Campaign end date
budget	DECIMAL(10,2)	Campaign budget
spent_amount	DECIMAL(10,2)	Amount spent
impressions	INT	Number of impressions
clicks	INT	Number of clicks
conversions	INT	Number of conversions
target_audience	TEXT	Target audience description
campaign_status	VARCHAR(20)	Campaign status

customer_sessions

Column	Type	Description
session_id	VARCHAR(50)	Unique session ID
customer_id	VARCHAR(50)	Foreign key to customers
session_start	TIMESTAMP	Session start time
session_end	TIMESTAMP	Session end time
device_type	VARCHAR(20)	Device used
referrer_source	VARCHAR(100)	Traffic source
pages_viewed	INT	Number of pages viewed
products_viewed	INT	Number of products viewed
conversion_flag	BOOLEAN	Whether session led to conversion
cart_abandonment_flag	BOOLEAN	Whether cart was abandoned

2. Batch File Ingestion

In batch mode, you upload data files into Pulse's storage, and the system processes them in one go.

Supported File Formats

The system accepts the following file formats:

Format	Extension	One or Multiple Tables?	Notes
CSV	.csv	One table per file	Simple, recommended. One file = one table
Excel	.xlsx, .xls	One sheet = one table	A single-sheet Excel is one table. Multiple sheets = multiple tables
Parquet	.parquet	One table per file	Columnar format, efficient for large data
JSON	.json	Both supported	Auto-detects structure — single array or multi-table object

Invalid files — Files with unsupported extensions (.txt, .pdf, .png, .docx, etc.) will be ignored and not processed.

Structure Requirements by File Type

CSV Files (.csv)

The simplest option. Each CSV file contains data for **one table only**. The file name (without extension) tells the system which table it belongs to.

```
customers.csv → maps to the "customers" table
orders.csv    → maps to the "orders" table
```

The first row must be a header with column names. Example:

```
customer_id,name,email,city,country
C001,John Doe,john@example.com,New York,United States
```

You do **not** need any special column naming for normal single-table CSV files. The system identifies the table from the file name and processes it automatically.

Fallback — the `table__column` format: The system has a fallback mechanism for cases where the file name cannot be matched to any canonical table. In this case, **all** columns must follow the `table__column` naming convention, where each column is prefixed with its canonical table name and a double underscore:

```
customers__customer_id,customers__name,orders__order_id,orders__amount
1,Alice,101,250
2,Bob,102,180
```

Important rules:

- Every single column must contain a double underscore (`__`) splitting the table name and column name
- The table name portion must be a valid canonical table name (e.g., `customers` , `orders` , `products`)
- The column name portion must be a valid canonical column for that table
- If any column violates these rules, the system will reject the file with an error

For most cases, separate files per table (named after the table) is the simplest approach and does not require this format.

Excel Files (.xlsx and .xls)

Excel files can be used in two ways:

Option 1 — One table per file (simplest): Upload a separate `.xlsx` or `.xls` file for each table. A single-sheet Excel file is treated as one table — the file name identifies the table.

```
customers.xlsx → one sheet named anything → maps to "customers" table
orders.xlsx   → one sheet named anything → maps to "orders" table
```

Option 2 — Multiple tables in one file (multi-sheet): Upload one `.xlsx` file with multiple sheets, each sheet representing a different table. The sheet name identifies the table.

```
ecommerce_data.xlsx:
├─ Sheet "customers" → maps to "customers" table
├─ Sheet "orders"    → maps to "orders" table
└─ Sheet "products"  → maps to "products" table
```

Rules for both options:

- Sheet names should match (or be similar to) a canonical table name
- Empty sheets are automatically skipped
- The first row of each sheet must be a header row with column names
- Sheet names can only contain letters, numbers, underscores, and hyphens

Parquet Files (.parquet)

Each Parquet file contains data for **one table only**. The file name (without extension) identifies the table.

```
customers.parquet → maps to the "customers" table
orders.parquet    → maps to the "orders" table
```

Columns are matched to the canonical schema by name — no special naming convention needed. If the file name cannot be matched to a canonical table, the same `table__column` fallback format described for CSV files applies (all columns must follow the format).

JSON Files (.json)

JSON files support three structures, all of which are automatically detected:

Format A — API-style (recommended for multiple tables):

```
{
  "tables": [
    {
      "table_name": "customers",
      "data": [
        {"customer_id": "C001", "name": "Alice"},
        {"customer_id": "C002", "name": "Bob"}
      ]
    },
    {
      "table_name": "orders",
      "data": [
        {"order_id": "101", "customer_id": "C001", "amount": 250}
      ]
    }
  ]
}
```

Format B — Nested table structure:

```
{
  "customers": [
    {"customer_id": "C001", "name": "Alice"},
    {"customer_id": "C002", "name": "Bob"}
  ],
  "orders": [
    {"order_id": "101", "customer_id": "C001", "amount": 250}
  ]
}
```

Format C — Single-table array (for one table only):

```
[
  {"customer_id": "C001", "name": "Alice"},
  {"customer_id": "C002", "name": "Bob"}
]
```

Important: When using Format A, the field name must be `table_name` (not `name`). Using `name` will cause a validation error.

3. Real-Time Streaming via Database URI (CDC)

In this mode, Pulse connects directly to your database using a connection URI. It uses a technology called **Change Data Capture (CDC)** to automatically capture every insert, update, and delete as they happen — without needing to constantly query your database.

The system works with the following databases (excluding Apache Cassandra):

Database	URI Scheme	Default Port
PostgreSQL	postgresql://	5432
MySQL	mysql://	3306
MariaDB	mariadb://	3306
MongoDB	mongodb://	27017
Microsoft SQL Server	mssql://	1433
Oracle	oracle://	1521
IBM Db2	db2://	50000
Vitess	vitess://	15991
Google Cloud Spanner	spanner://	N/A
Informix	informix://	9088

Connection URI Format

The general format for the connection URI is:

```
scheme://username:password@host:port/database
```

Examples:

- PostgreSQL: `postgresql://debezium_user:password@hostname:5432/ecommerce`
- MySQL: `mysql://debezium_user:password@db.example.com:3306/shop`
- MongoDB: `mongodb://debezium_user:password@mongo-host:27017/orders?replicaSet=rs0`
- SQL Server: `mssql://debezium_user:password@sqlserver.local:1433/sales`
- Oracle: `oracle://debezium_user:password@oracle-host:1521/ORCL`
- IBM Db2: `db2://debezium_user:password@db2-host:50000/ECOMDB`
- Vitess: `vitess://vtgate-host:15991/commerce`
- Google Cloud Spanner: `spanner://project-id/instance-id/database-id`
- Informix: `informix://debezium_user:password@informix-host:9088/stores`

Preparing Your Database for CDC

For Pulse to stream data from your database, your database administrator (DBA) needs to do a one-time setup. The exact steps depend on your database type. Below are the official documentation links and a brief summary of what is required for each supported database (excluding Cassandra).

PostgreSQL

What to do: Enable logical replication, create a dedicated user with replication privileges, and create a publication for the tables you want to stream.

Documentation:

<https://debezium.io/documentation/reference/stable/connectors/postgresql.html>

MySQL

What to do: Enable binary logging (binlog) with ROW format, create a user with REPLICATION SLAVE and REPLICATION CLIENT privileges.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/mysql.html>

MariaDB

What to do: Enable binary logging with ROW format, create a user with REPLICATION SLAVE and REPLICATION CLIENT privileges.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/mariadb.html>

MongoDB

What to do: Configure MongoDB as a replica set (even a single-node replica set works), create a user with read permissions on the database and local/oplog.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/mongodb.html>

Microsoft SQL Server

What to do: Enable CDC on the database and each table you want to stream, ensure SQL Server Agent is running, create a user with db_datareader role.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/sqlserver.html>

Oracle Database

What to do: Enable ARCHIVELOG mode, enable supplemental logging, install LogMiner, create a user with LogMiner and SELECT privileges.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/oracle.html>

IBM Db2

What to do: Enable SQL Replication, create ASNCDC schema, register tables for capture, start the ASN Capture service, create a user with SELECT privileges.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/db2.html>

Vitess

What to do: Enable VStream on vtgate, create a user with SELECT and REPLICATION SLAVE privileges.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/vitess.html>

Google Cloud Spanner

What to do: Create a change stream, create a GCP service account with spanner.databaseReader and spanner.changeStreamReader roles, download the JSON key file.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/spanner.html>

Informix

What to do: Enable CDC on the database and each table, create a user with SELECT privileges.

Documentation: <https://debezium.io/documentation/reference/stable/connectors/informix.html>

4. Real-Time Streaming via API

In API mode, Pulse polls your API endpoint at regular intervals to fetch data. You provide the URL of your API, and Pulse handles the rest.

Preparing Your API Endpoint

Your API endpoint must:

1. Be accessible over HTTP/HTTPS from Pulse's network
2. Return a JSON response in the expected format (see below)

3. Optionally support the `updated_since` query parameter for incremental polling (recommended)

Expected API Response Format

Pulse expects your API to return JSON in the following structure:

```
{
  "tables": [
    {
      "table_name": "customers",
      "data": [
        {"customer_id": "C001", "name": "John Doe", "email": "john@example.com"},
        {"customer_id": "C002", "name": "Jane Smith", "email": "jane@example.com"}
      ]
    },
    {
      "table_name": "orders",
      "data": [
        {"order_id": "101", "customer_id": "C001", "amount": 250.00},
        {"order_id": "102", "customer_id": "C002", "amount": 180.00}
      ]
    }
  ]
}
```

Key rules:

- The root object must have a `tables` key containing an array
- Each object in the array must have:
 - **table_name** (string): The name of the table (must match or be similar to a canonical table name)
 - **data** (array): An array of record objects, where each record is a flat JSON object with column names as keys
- **table_name** must not be empty and can only contain letters, numbers, underscores, or hyphens
- Table names are automatically converted to lowercase
- The field name must be `table_name` (not `name`) — using `name` will cause a validation error

Incremental Polling (Recommended)

To avoid fetching the entire dataset on every poll, Pulse supports **incremental polling**. After the first poll (which fetches all data), Pulse appends the query parameter `?updated_since=<ISO8601_timestamp>` to every subsequent request:

```
GET https://your-api.com/data?updated_since=2026-03-10T10:30:00Z
```

Your API should use this timestamp to filter and return only records that were created or modified after that time. This makes polling much more efficient.

Example of how to implement this in your API:

```
from flask import Flask, jsonify, request
from datetime import datetime

app = Flask(__name__)

@app.route('/api/data')
def get_data():
    updated_since_str = request.args.get('updated_since')
    updated_since = None
    if updated_since_str:
        updated_since = datetime.fromisoformat(
            updated_since_str.replace('Z', '+00:00')
        )

    customers = get_customers(updated_since=updated_since)
    orders = get_orders(updated_since=updated_since)

    return jsonify({
        "tables": [
            {"table_name": "customers", "data": customers},
            {"table_name": "orders", "data": orders},
        ]
    })
```

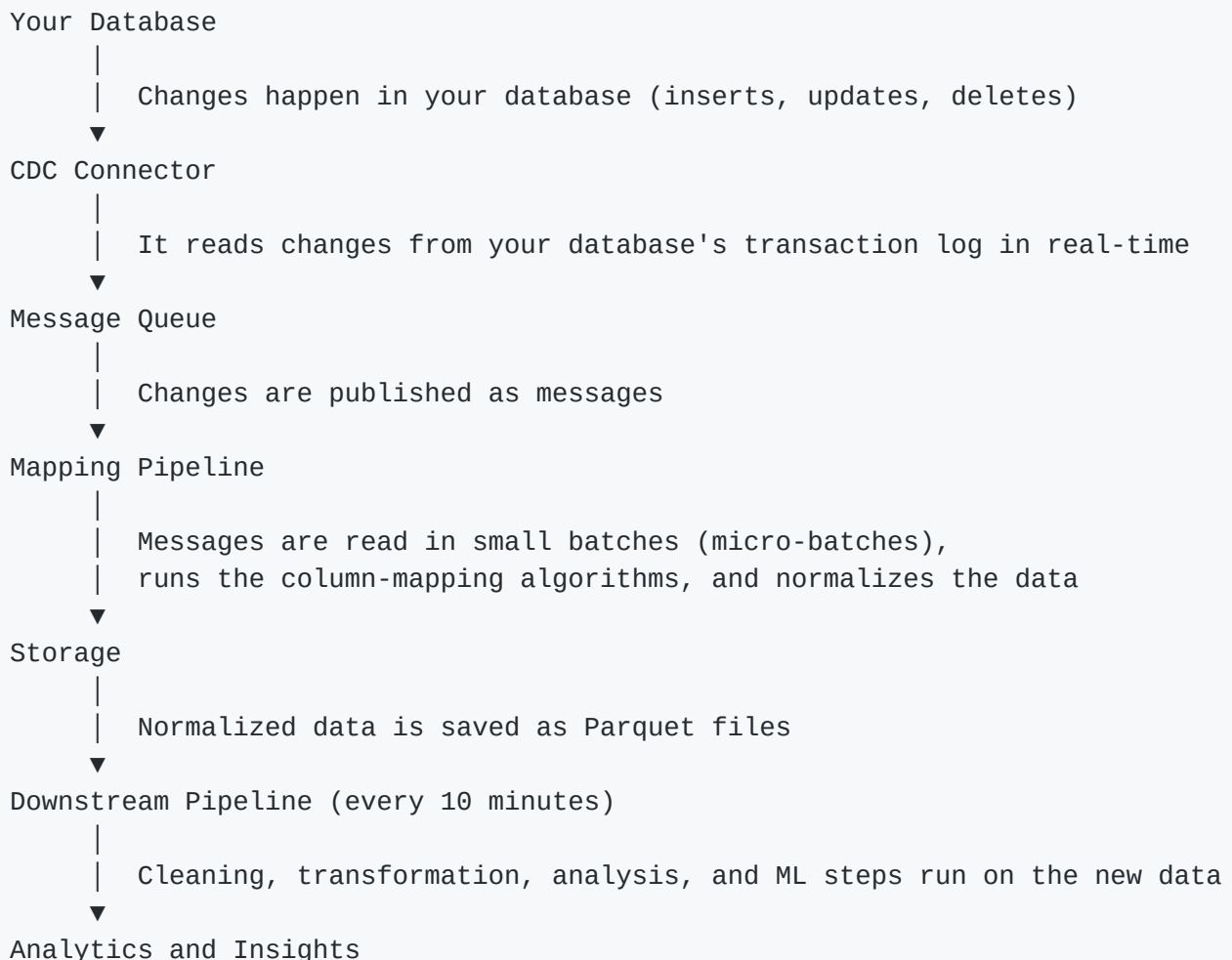
If your API does not support the `updated_since` parameter, Pulse will simply fetch all records on every poll. The system handles this gracefully with deduplication.

5. How the Ingestion System Works

This section explains what happens behind the scenes when Pulse ingests data in real-time, whether from a database or an API.

Real-Time Streaming from a Database (CDC)

Here is the step-by-step flow:

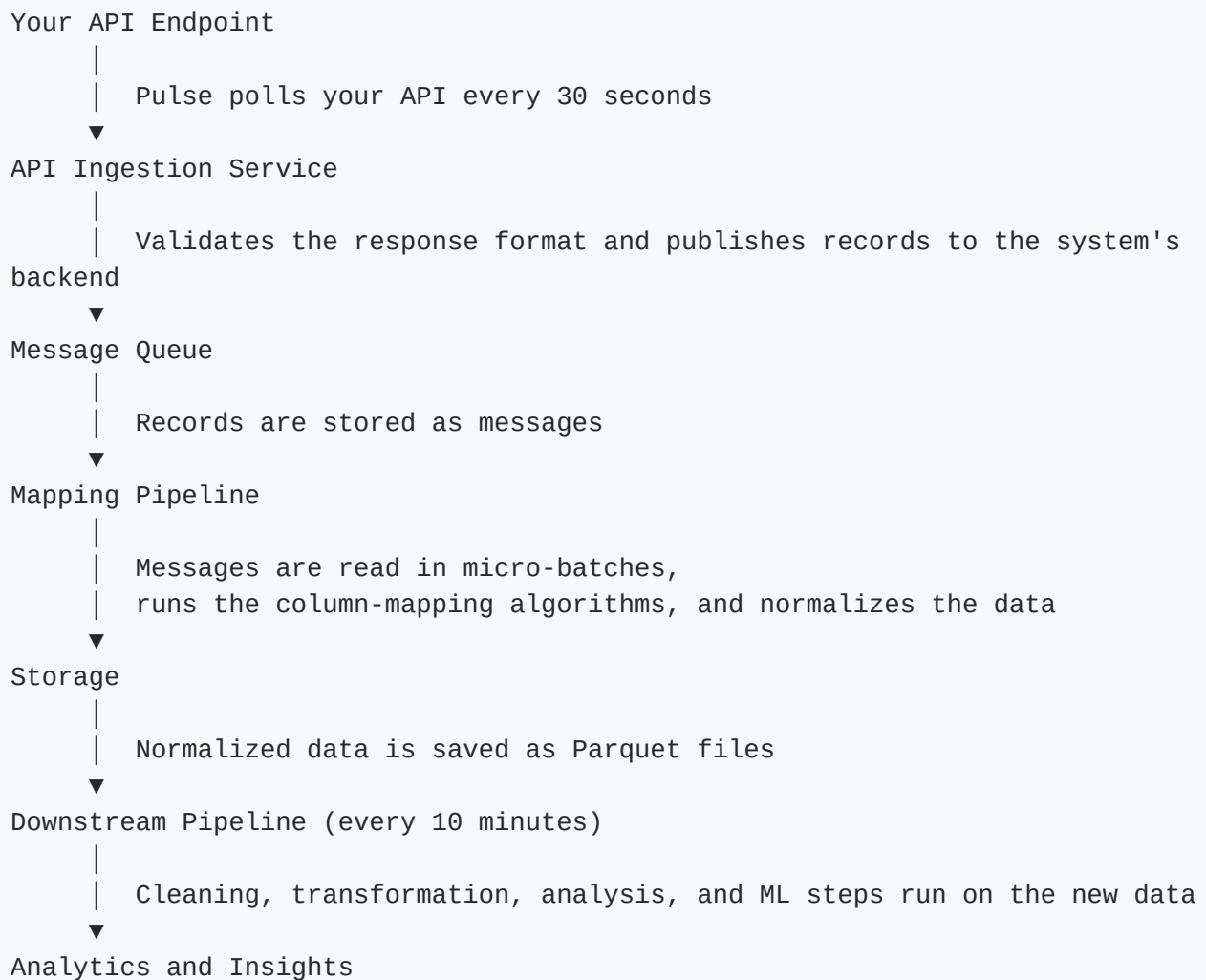


Key points:

- As soon as a change happens in your database, CDC captures it immediately
- The captured change is sent to the message queue within seconds
- Micro-batches are processed continuously — each micro-batch starts processing as soon as the previous one finishes (no waiting). Each micro-batch is capped at roughly 50,000 messages, keeping individual batches small and fast
- The downstream batch pipeline (cleaning, transformation, analysis, ML) runs **every 10 minutes** and picks up all new data that arrived in the "mapped/" folder since the last run

Real-Time Streaming from an API

Here is the step-by-step flow:



Key points:

- Pulse polls your API endpoint every **30 seconds** by default (configurable)
- The first poll fetches all data (baseline). Subsequent polls only fetch records that changed since the last poll (if your API supports the `updated_since` parameter)
- Micro-batches are processed continuously, just like in database mode
- The downstream batch pipeline runs **every 10 minutes** for both modes

Timing Summary

What happens	How often
CDC capture (Database mode)	In real-time, as changes happen
API polling (API mode)	Every 30 seconds
Micro-batch processing	Continuously (as fast as each batch finishes)
Downstream batch pipeline (cleaning, transformation, analysis, ML)	Every 10 minutes

What Happens on the First Connection (Initial Load)

The very first time Pulse connects to your database or API:

1. **Database mode:** Pulse performs an initial bulk load, reading all existing data from your database tables in chunks of approximately 200,000 rows each. At the same time, CDC starts tracking the transaction log from the current position (no duplicate data). After the bulk load completes, the batch mapping pipeline processes all the ingested data and saves the results.
2. **API mode:** Pulse polls your API for a set duration (default 2 minutes) to collect an initial batch of data. After polling completes, the batch mapping pipeline processes all the data and saves results.
3. After the initial load, the system switches to continuous streaming mode — new changes are processed in real-time as described above.

About Missing Columns and Manual Mapping

Pulse's mapping pipeline uses seven different algorithms (fuzzy matching, NLP, semantic similarity, and more) to automatically match your column names to the canonical schema columns. If some columns cannot be automatically matched:

- Those columns are flagged as "missing" and the data is saved to a temporary location
- The frontend displays which columns could not be mapped
- You can provide manual mappings through the frontend, telling the system which source column corresponds to which canonical column
- After you confirm the manual mappings, the system applies them, and all future streaming data will be mapped correctly automatically

Once the initial mapping is confirmed, the system remembers these mappings and applies them to all future data automatically.
